Definiendo ButtonLike recetas

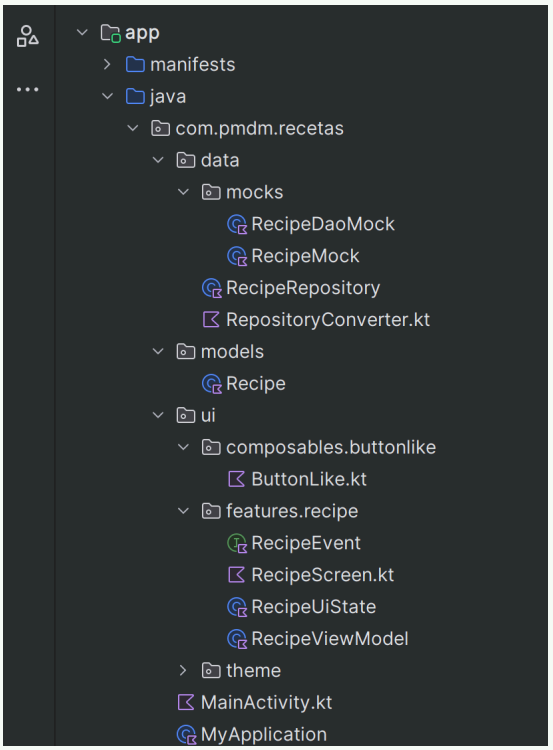
Descargar estos apuntes

Ejercicio

Vamos a añadir al ejercicio `Card` donde se visualiza la información de una receta la lógica para elevar el estado al `ViewModel`. En el *preview* se muestra el card siguiendo los convenios de Material Design 3 (M3).



Siguiendo la arquitectura propuesta para nuestras aplicaciones, la estructura de paquetes y contenido de los mismos con sus correspondientes archivos `.kt` es la siguiente:



Paso 1: Definiendo el `model`

En el paquete **`com.pmdm.recetas`** vamos a crear el paquete **`models`** donde según la arquitectura propuesta en el curso, se definirán los modelos de datos de nuestra **`app`**. Dentro crearemos el fichero **`Recipe.kt`**

```
data class Recipe(
    val name: String,
    val description: String,
    val chef: String,
    val urlPhoto: String?,
    val numLikes: Int,
    val iLike: Boolean
)
```

Estamos simplificando bastante la solución del ejercicio, por eso vamos a suponer que el atributo **`iLike`** pertenece al objeto receta. En una aplicación más desarrollada este atributo pertenecería a una relación existente entre el usuario concreto y la receta.

`numLikes` es el total de likes que tiene la receta y que guardaremos en la BD. Se verá afectado si **`iLike`** es `true` o `false`. Si es `true` el usuario ha dado like a la receta y por tanto el total de likes se incrementa en 1. Si es `false` el usuario ha retirado su like y por tanto el total de likes se decrementa en 1.

Paso 2: Definiendo el paquete `data.mocks`

En el paquete **`com.pmdm.recetas.data.mocks`** vamos a crear dos archivos: **`RecipeDaoMock.kt`** (una `class`) y **`RecipeMock.kt`** (un `data class`).

La clase **`RecipeDaoMock.kt`** solo contendrá una objeto de tipo receta con los métodos que me permitirán obtener la receta y actualizar la receta (cuando se modifique a través de la acción del `ButtonLike`):

> **Nota**
>
> Debes crear dentro de del servidor de **nginx** un directorio **`recetas`** y dentro de él un archivo **`magdalenas.png`** con la imagen de las magdalenas que puedes descargar de **este enlace**.

```
class RecipeDaoMock {
    private var recipe =
        RecipeMock(
            name = "Magdalenas de la abuela",
            description = "Fabulosas magdalenas con pepitas de chocolate y un suave sabor a naranja.",
            chef = "Carlos Arguiñano",
            urlPhoto = "http://10.0.2.2/recetas/magdalenas.png",
            numLikes = 8,
            iLike = false
        )

    fun get(): RecipeMock = recipe
    fun updateRecipe(recipeRemote: RecipeMock) {
        recipe = recipeRemote
    }
}
```

El' **`data class`** **`RecipeMock.kt`** es idéntico en su definición a la clase de nuestro modelo **`Recipe.kt`**.

Dentro del paquete **`data`**, pero no dentro de **`data.mocks`**, definimos los archivos **`RecipeRepository.kt`** (una `class`) y **`RepositoryCoverter.kt`** con los conversores entre **`Recipe`** y **`RecipeMock`** únicamente.

Con los conocimientos adquiridos en ejercicios previos debéis poder definir estos dos elementos.

Paso 3: Definiendo el estado de la vista

Para manejar el estado de nuestra vista definiremos el archivo **`RecipeUiState.kt`**:

```
data class RecipeUiState(
    val name: String,
    val description: String,
    val chef: String,
    var urlPhoto: String?,
    val numLikes: Int,
    val iLike: Boolean
)
```

y sus conversores entre **`RecipeUiState`** y **`Recipe`** como métodos de extensión en el archivo **`RecipeUiState.kt`**:

Con todos estos elementos definidos, es el momento de que definái s el **`ViewModel`**.

A tener en cuenta en esta definición es que a través de `RecipeEvent` vamos a gestionar todos los eventos sobre el `Card`. En nuestro caso solo será uno, pero mantenemos esta estructura de funcionamiento, que ya hemos utilizado en ejemplos anteriores, para fijar en nuestra mente esta forma de procesar eventos.

Pista

El siguiente código es un *preview* de la pantalla de recetas, donde se define el estado de la receta y el evento que se va a procesar. Te puede servir de pista para definir el `ViewModel` y el `RecipeEvent`.

```
@Composable
fun RecetaPreview() {
    var recipeState by remember {
        mutableStateOf(RecipeRepository().get().toRecipeUiState())
    }
    val onRecipeEvent = { e: RecipeEvent ->
        when (e) {
            is OnClickButtonLike -> {
                recipeState = // TODO: Actualizar el estado de la receta con
                              // el nuevo valor de iLike y numLikes
            }
        }
    }

    RecetasTheme {
        Column(
            modifier = Modifier.padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            RecipeScreen(
                recipeState = recipeState,
                onRecipeEvent = onRecipeEvent
            )
        }
    }
}
```